The Systems Behind Modern AI

Why and how does hardware designed for 3D games power modern AI

Sam Ly

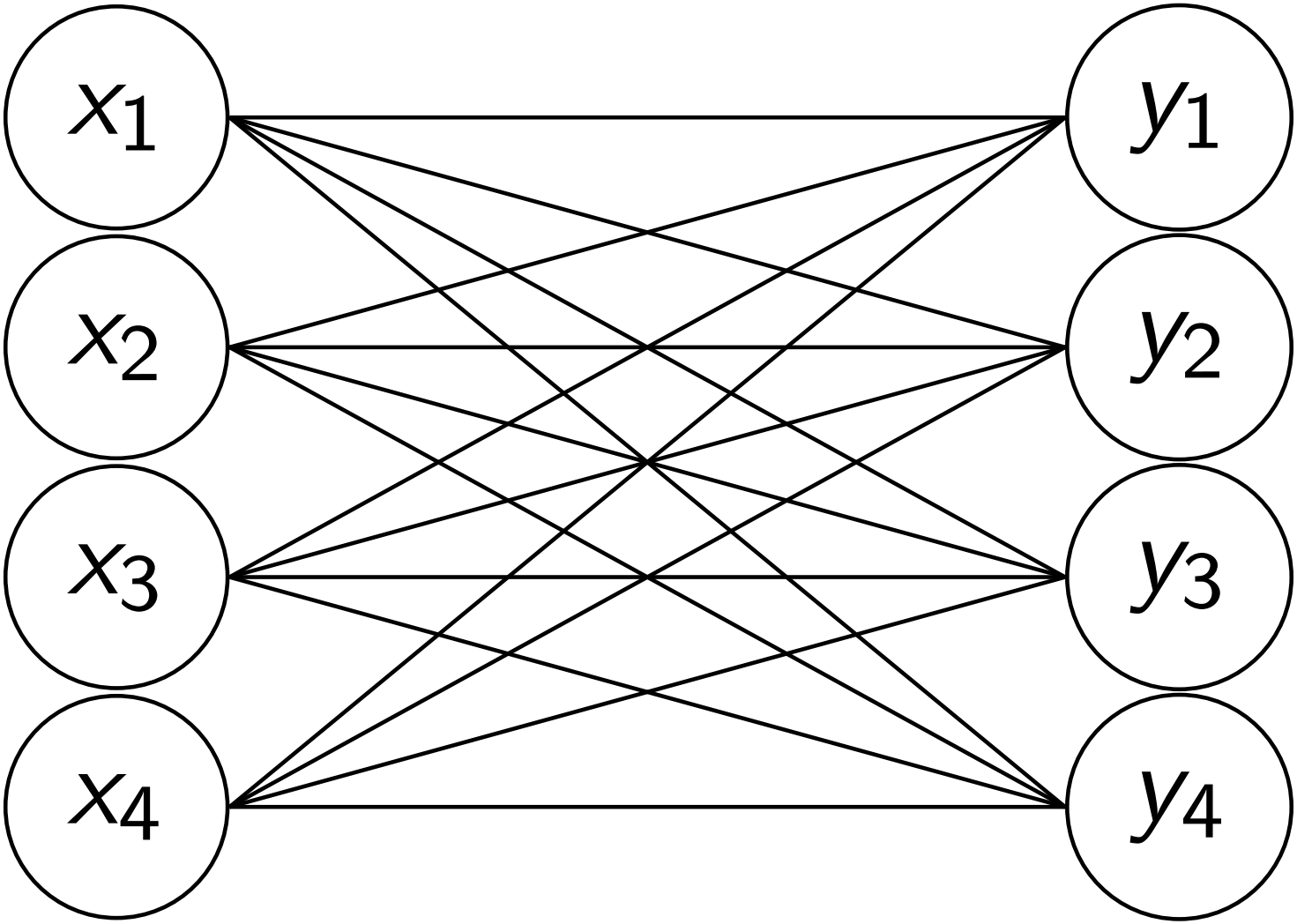
Cal Poly Pomona

Spring 2026

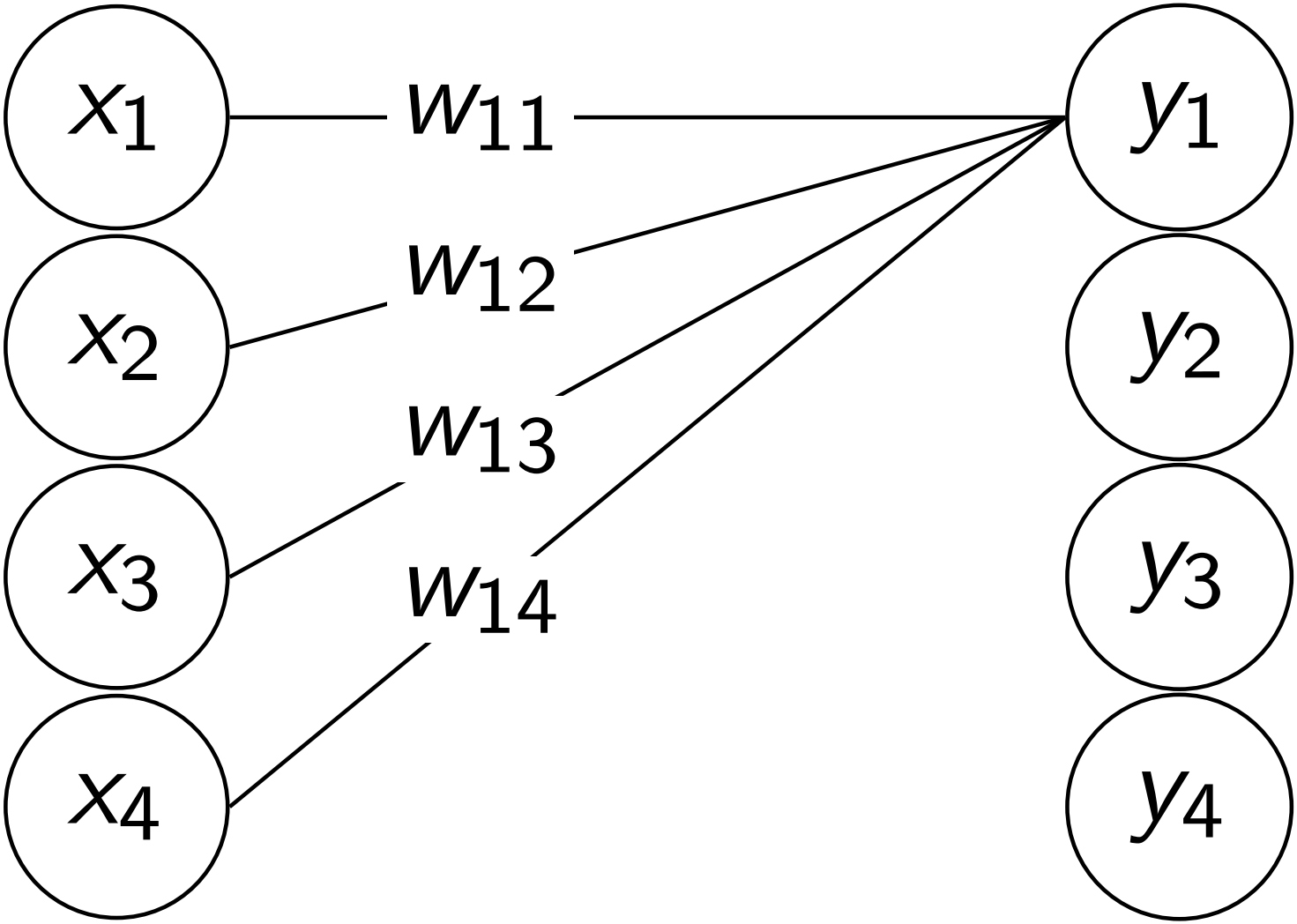
Table of Contents

1. Neural Networks and the Matrix Multiplication
2. Why GPUs?
3. Optimizing **Ge**neral **M**atrix **M**ultiplication (GEMM) on CUDA

Basic Neural Net (MLP/FFN)

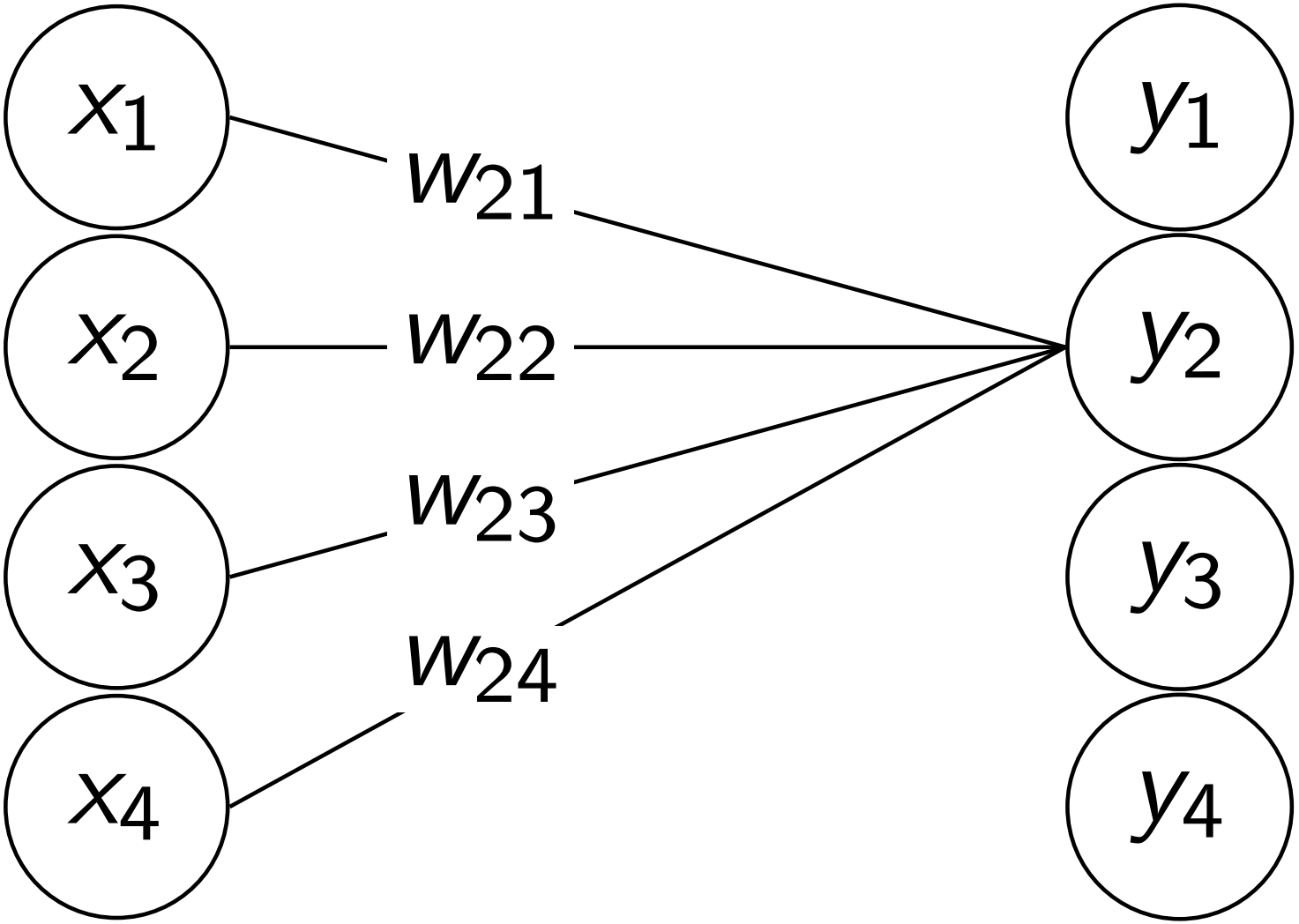


Basic Neural Net (MLP/FFN)



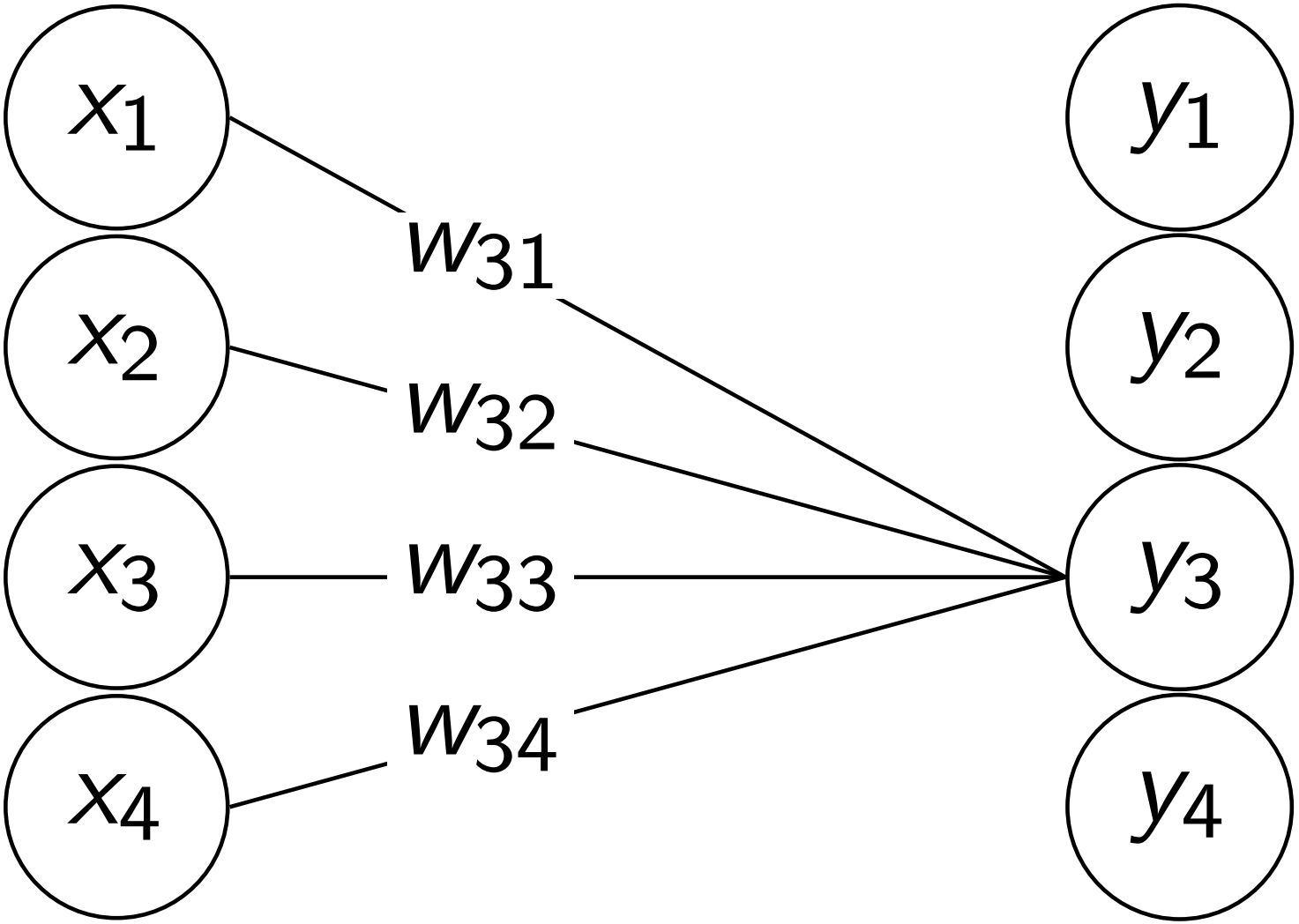
$$y_1 = x_1 w_{11} + x_2 w_{12} + x_3 w_{13} + x_4 w_{14}$$

Basic Neural Net (MLP/FFN)



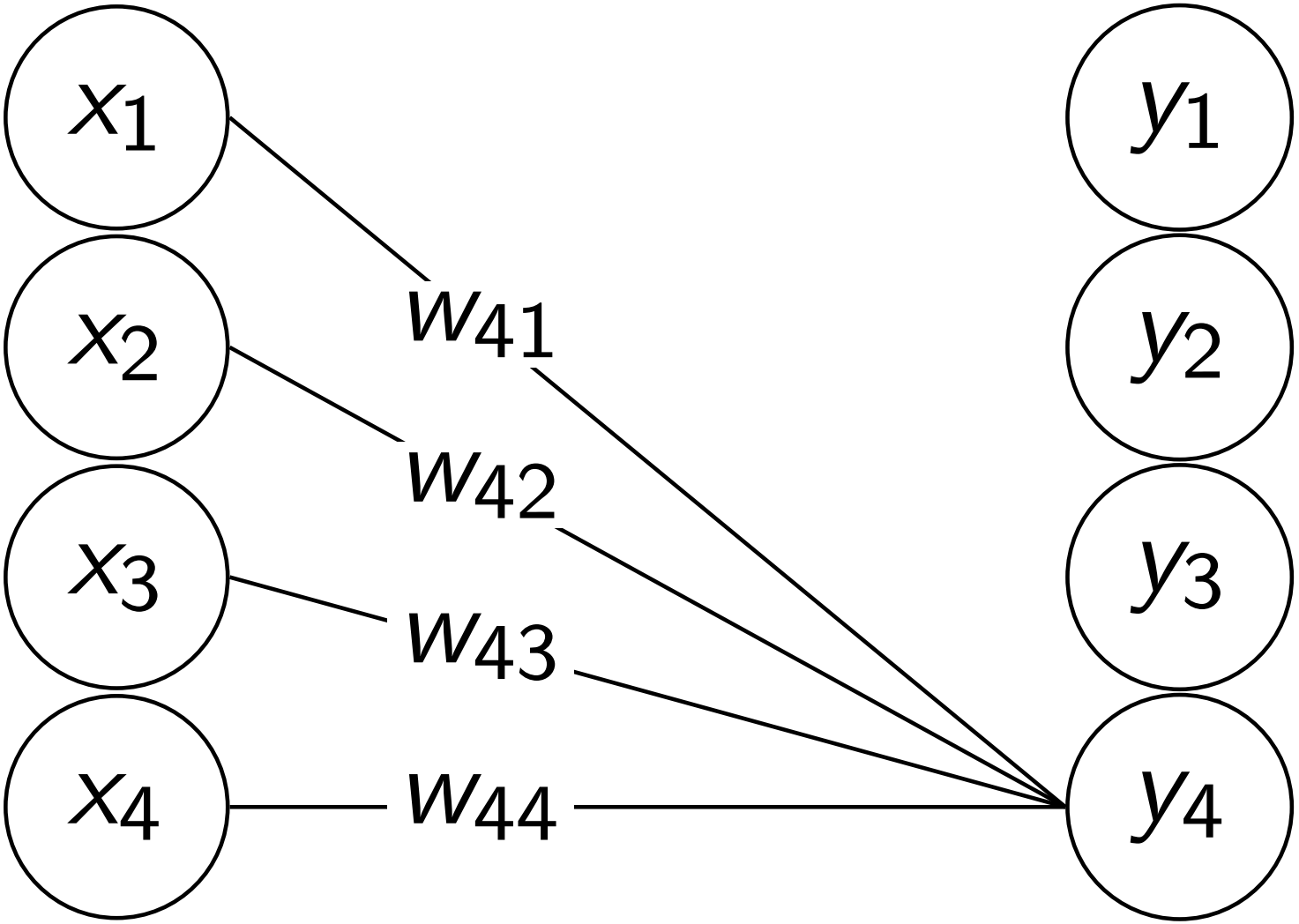
$$y_2 = x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + x_4 w_{24}$$

Basic Neural Net (MLP/FFN)



$$y_3 = x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + x_4 w_{34}$$

Basic Neural Net (MLP/FFN)



$$y_4 = x_1 w_{41} + x_2 w_{42} + x_3 w_{43} + x_4 w_{44}$$

Linear Algebra Interpretation

Here, notice that for each y_i, we are taking a weighted sum of the x elements. That is,

$$y_i = x_1 w_{i1} + x_2 w_{i2} + x_3 w_{i3} + x_4 w_{i4}.$$

Arranging the y_i into a vector, we get

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} \\ w_{21} & w_{22} & w_{23} & w_{24} \\ w_{31} & w_{32} & w_{33} & w_{34} \\ w_{41} & w_{42} & w_{43} & w_{44} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}.$$

Linear Algebra Interpretation

We can generalize the number of x's and the number of y's.

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

We say, n is the *input dimension* and m is the *output dimension*.

Computing with multiple x vectors.

Suppose we have multiple x vectors, for which we want to pass through our neural network. We can stack the multiple x vectors in the **batch**, which we compute with a matrix multiplication.

$$\begin{bmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(B)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ y_m^{(1)} & y_m^{(2)} & \cdots & y_m^{(B)} \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \cdots & w_{mn} \end{bmatrix} \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(B)} \\ x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(B)} \\ \vdots & \vdots & \ddots & \vdots \\ x_n^{(1)} & x_n^{(2)} & \cdots & x_n^{(B)} \end{bmatrix}.$$

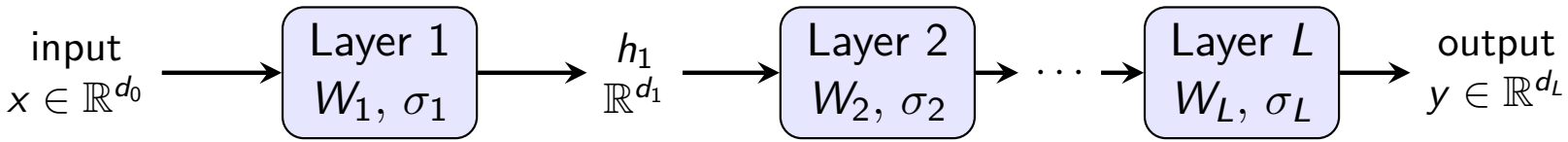
We write this succinctly as $Y = WX$, where $Y \in \mathbb{R}^{m \times B}$, $X \in \mathbb{R}^{n \times B}$, and $W \in \mathbb{R}^{m \times n}$.

Yippee!

We have successfully mapped the batched computation of a **linear layer** a matrix multiplication!

Composing Layers

We can view W as a function that takes in an n-dimensional vector and maps it to an m-dimensional vector. $W : \mathbb{R}^n \to \mathbb{R}^m$. We create deep neural networks by *composing* these transformations, along with **nonlinearities** σ_i.

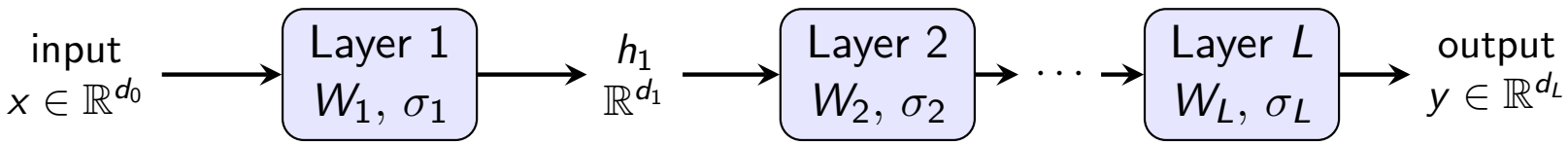


$$h_0 = x, \qquad h_\ell = \sigma_\ell(W_\ell h_{\ell-1}), \qquad y = h_L$$

Remark

Notice that this figure doesn't show the *batch* dimension. However, if you work through the dimension mappings, the *batch* dimension B will be unaffected by this composition.

Limitations for Neural Network Architecture



$$h_0 = x, \qquad h_\ell = \sigma_\ell(W_\ell h_{\ell-1}), \qquad y = h_L$$

Architectures such as **Transformers** and **CNNs** have connectivity patterns more complex than a simple composition of layers. Nevertheless, linear transformations remain a fundamental computational primitive in neural network architectures.

A fundamental building block

A linear transformation may seem like a narrow operation to study, but modern neural networks perform it repeatedly and at enormous scale. Its efficiency therefore has an outsized effect on training and inference performance. The rest of this presentation explores the hardware and techniques that make these operations efficient.

Takeaways

1. A linear layer applied to a batch of inputs can be expressed as a single matrix multiplication, $Y = WX$.
2. Deep networks compose these linear transformations with nonlinearities. More complex architectures change how layers are connected, but linear transformations remain a fundamental building block.
3. **Matrix multiplication therefore captures a large and reusable part of neural network computation.**

Why is this representation useful?

It separates what the network computes from how the hardware executes it. Instead of optimizing every architecture from scratch, we can focus on one common computational primitive. Next, we will examine how GPUs execute matrix multiplications efficiently.

Table of Contents

1 Neural Networks and the Matrix Multiplication

2 Why GPUs?

3 Optimizing **Ge**neral **M**atrix **M**ultiplication (GEMM) on CUDA

From graphics pipeline to AI accelerator

- **1999—2003:** fixed-function graphics gives way to programmable vertex and pixel shaders.
- **2004:** Brook for GPUs demonstrates a high-level stream-programming model on top of graphics APIs.
- **2006—2007:** GeForce 8800 introduces a unified shader architecture; NVIDIA releases CUDA publicly in 2007.
- **2010:** Fermi adds a coherent cache hierarchy, ECC support, and much stronger support for C++ and scientific computing.
- **2017:** Volta introduces Tensor Cores for mixed-precision matrix multiply–accumulate.
- **2020—2024:** Ampere broadens Tensor Core formats; Hopper adds the FP8 Transformer Engine; Blackwell adds further low-precision and AI-specific hardware.

Demand for 3D graphics

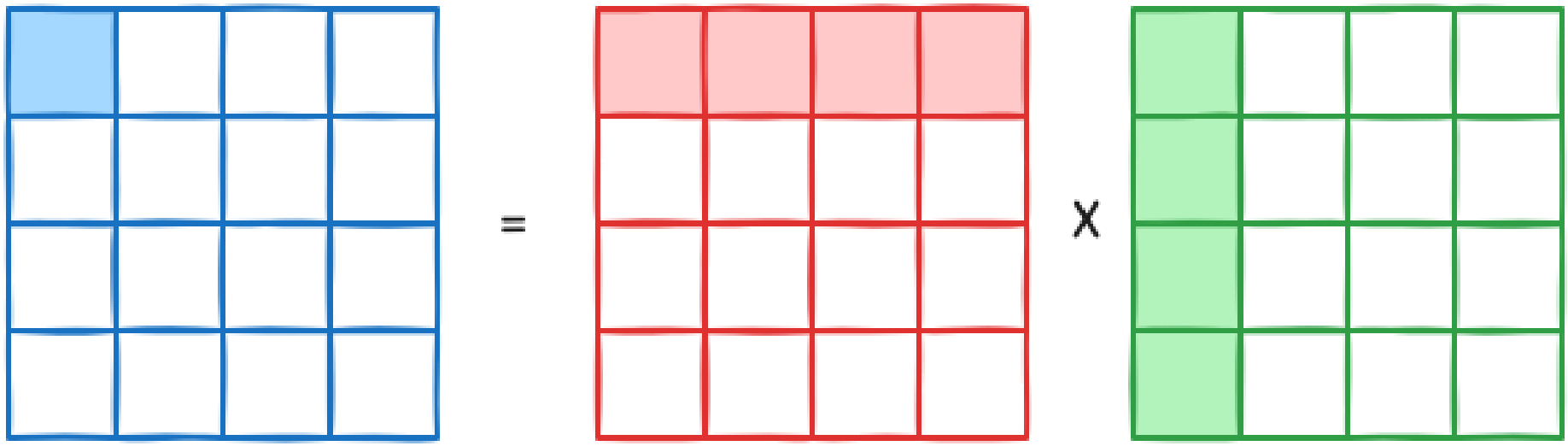
GPU's came about due to the increase in computational power needed to render 3D games in real time. This requires *millions* of FLOP's per second. CPU's could not keep up.

Parallelism

Rendering graphics has a nice property that we can take advantage of: **parallelism**.
Each pixel on the screen doesn't really care any of the other pixels. Furthermore, the computation that is required for each pixel isn't that complex either. The difficulty in rendering graphics is the volume of computations.

Computing a Matrix Multiplication

Let A and B be two 4×4 matrices. The matrix multiple $C = AB$ results in another 4×4 matrix.



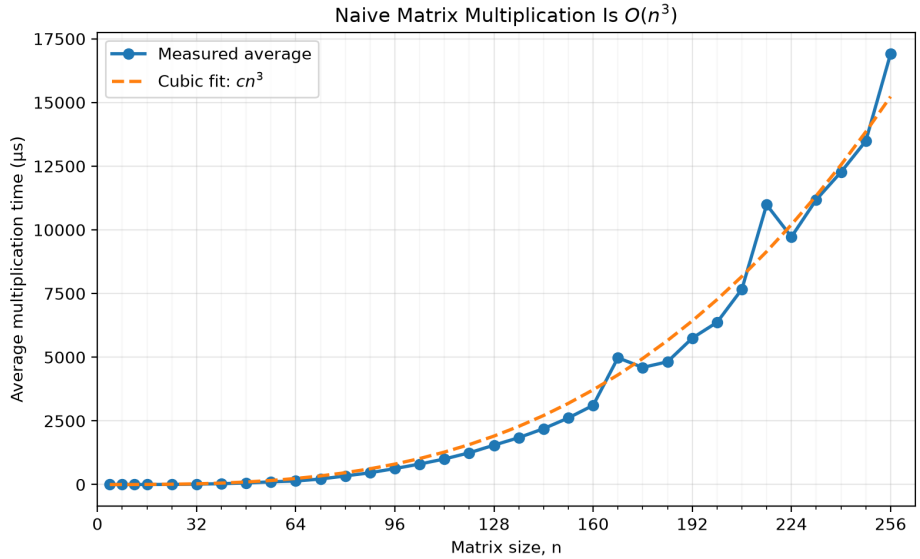
The operations are very simple, but the number of operations grow on the order of $\mathcal{O}(n^3)$, where n is the dimensions of the matrices.

Remark

Linear algebra and matrix multiplications show up everywhere in scientific computing, so computing them efficiently has always been in high demand.

Runtime of a matrix multiplication on a CPU

```
for (std::size_t i = 0; i < n; ++i) {
  for (std::size_t j = 0; j < n; ++j) {
    double sum = 0.0;
    for (std::size_t k = 0; k < n; ++k) {
      sum += a[i * n + k] * b[k * n + j];
    }
    c[i * n + j] = sum;
  }
}
```



Matrix Multiplication is Parallel

Each element of the output is independent. The results of C_{11} does not effect the results of C_{12}, or any other element.

Remark

If matrix multiplication is a **parallel** task, and GPU's were designed for the **parallel** task of rendering graphics, can we repurpose GPU's to do matrix multiplication?

Matrix Multiplication is Parallel

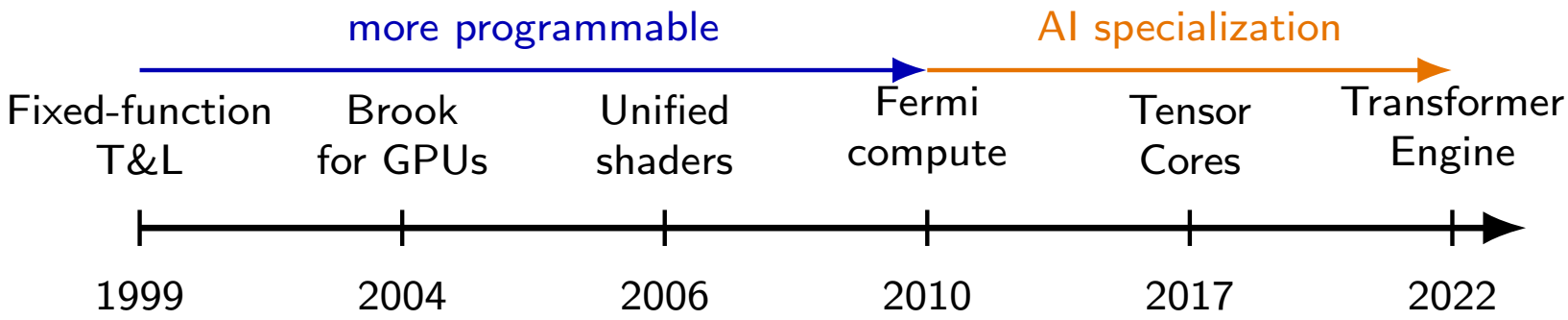
Each element of the output is independent. The results of C_{11} does not effect the results of C_{12}, or any other element.

Remark

If matrix multiplication is a **parallel** task, and GPU's were designed for the **parallel** task of rendering graphics, can we repurpose GPU's to do matrix multiplication? (Yes.)

What shaped modern GPU architecture?

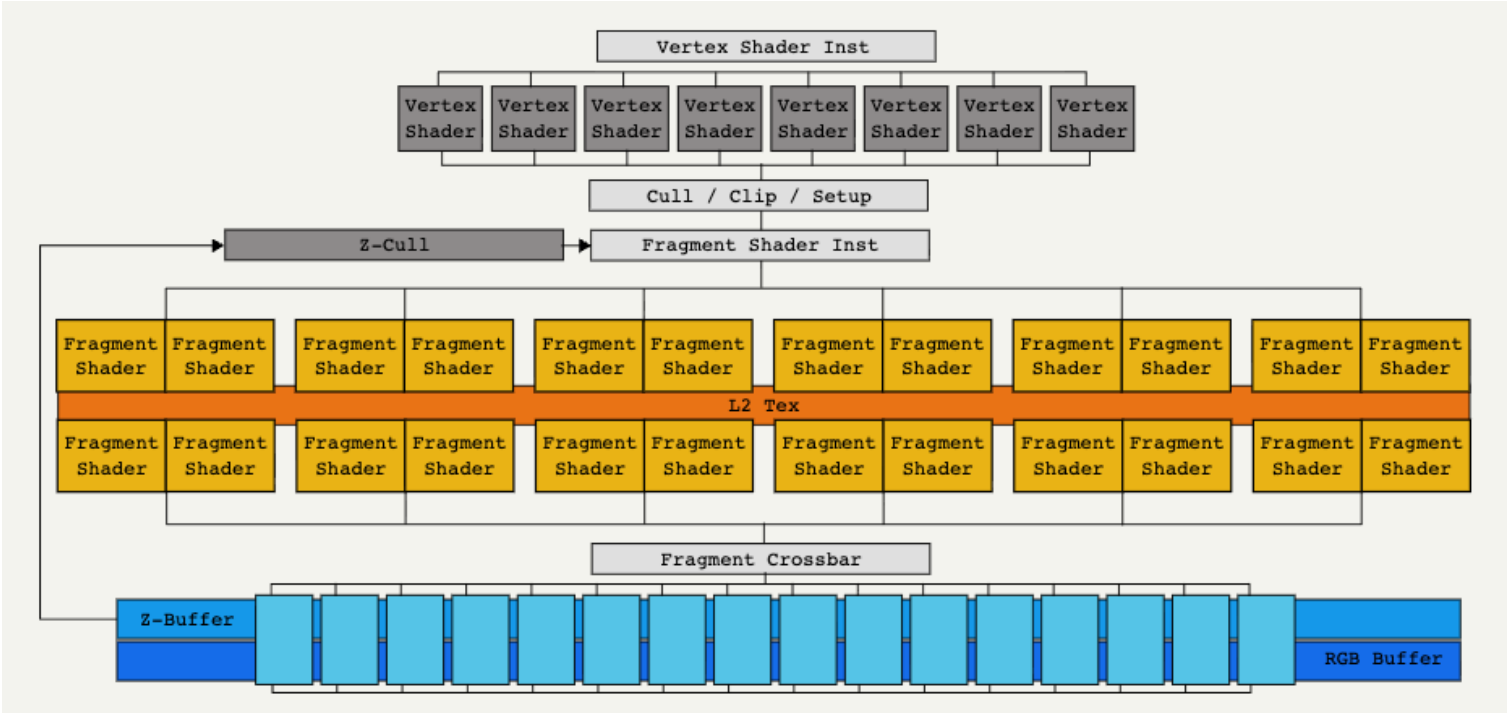
New workloads cause generalization/programmability, established workloads cause specialization.



What shaped modern GPU architecture?

$<$2006: graphics first

- GeForce 256 (1999) accelerated fixed-function transform and lighting.
- Later chips added programmable vertex and pixel shaders, but kept distinct stages and resources.



The G71 architecture (2005). Notice the distinct **hardware** elements per graphics task.

What shaped modern GPU architecture?

2004: GPGPU before CUDA. Demand was always there.

- Brook for GPUs compiled a stream language to graphics APIs.
- It showed useful non-graphics workloads could exploit GPU throughput, despite an awkward programming model.

Brook for GPUs: Stream Computing on Graphics Hardware

Ian Buck Tim Foley Daniel Horn Jeremy Sugerman Kayvon Fatahalian Mike Houston Pat Hanrahan
Stanford University

Abstract

In this paper, we present Brook for GPUs, a system for general-purpose computation on programmable graphics hardware. Brook extends C to include simple data-parallel constructs, enabling the use of the GPU as a streaming co-processor. We present a compiler and runtime system that abstracts and virtualizes many aspects of graphics hardware. In addition, we present an analysis of the effectiveness of the GPU as a compute engine compared to the CPU, to determine when the GPU can outperform the CPU for a particular algorithm. We evaluate our system with five applications, the SAXPY and SGEMV BLAS operators, image segmentation, FFT, and ray tracing. For these applications, we demonstrate that our Brook implementations perform comparably to hand-written GPU code and up to seven times faster than their CPU counterparts.

CR Categories: I.3.1 [Computer Graphics]: Hardware Architecture—Graphics processors D.3.2 [Programming Languages]: Language Classifications—Parallel Languages

Keywords: Programmable Graphics Hardware, Data Parallel Computing, Stream Computing, GPU Computing, Brook

modern hardware. In addition, the user is forced to express their algorithm in terms of graphics primitives, such as textures and triangles. As a result, general-purpose GPU computing is limited to only the most advanced graphics developers.

This paper presents *Brook*, a programming environment that provides developers with a view of the GPU as a streaming coprocessor. The main contributions of this paper are:

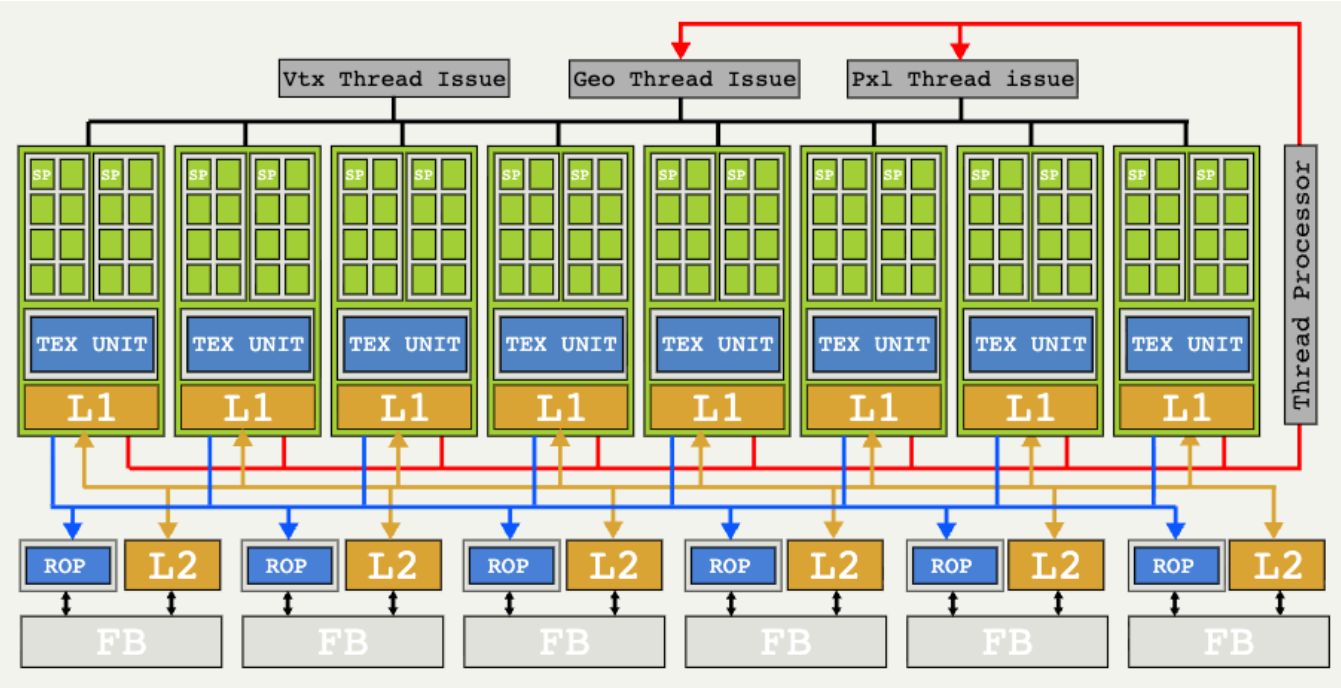
- The presentation of the Brook stream programming model for general-purpose GPU computing. Through the use of streams, kernels and reduction operators, Brook abstracts the GPU as a streaming processor.
- The demonstration of how various GPU hardware limitations can be virtualized or extended using our compiler and runtime system; specifically, the GPU memory system, the number of supported shader outputs, and support for user-defined data structures.
- The presentation of a cost model for comparing GPU vs. CPU performance tradeoffs to better understand under what circumstances the GPU outperforms the CPU.

Buck et al., "Brook for GPUs," *ACM TOG* 23(3), 2004.

What shaped modern GPU architecture?

2006-2007: increasing complexity necessitates unified compute

- G80 ran vertex, geometry, and pixel work on unified pool of **Streaming Multiprocessors** (SM's).
- Publicly released in 2007, CUDA exposed this parallel machine without graphics API tricks.

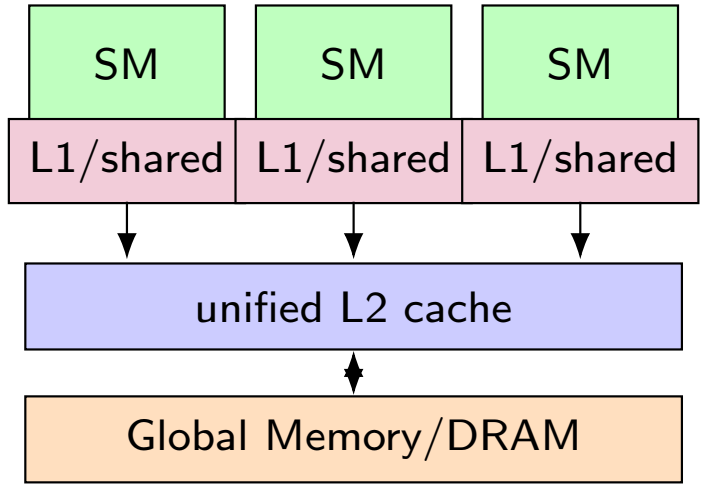


G80 block diagram. Source: NVIDIA, *GeForce 8800 GPU Architecture Overview*, 2006.

What shaped modern GPU architecture?

2010: Fermi makes compute first-class

- Continues trend of general cores, scales horizontally.
- A unified L2 cache serves all SMs, establishes the heart of modern GPU memory hierarchy.
- Features for HPC like ECC and double precision FP.



Conceptual memory hierarchy. Based on NVIDIA, *Fermi Compute Architecture Whitepaper*, 2009.

Takeaways

1. The parallel nature of graphics rendering lends nicely to matrix multiplication
2. GPU hardware continuously evolves with demand. (Explored more later.)

Table of Contents

1. Neural Networks and the Matrix Multiplication
2. Why GPUs?
3. Optimizing **Ge**neral **M**atrix **M**ultiplication (GEMM) on CUDA

Why GEMM?

Single-precision general matrix multiplication (SGEMM)

$$C \leftarrow \alpha AB + \beta C$$

- Setting $\alpha = 1$ and $\beta = 0$ gives ordinary matrix multiplication: $C = AB$.
- Other values let one call scale the product and accumulate it into an existing matrix.
- Matrix multiplication accounts for much of the work in neural-network training and inference.
- Our goal is unchanged math with less data movement and more useful work per thread.

Source: Simon Boehm, "How to Optimize a CUDA Matmul Kernel" (2022).

A closer look at CUDA

Execution hierarchy

- A kernel launch creates a **grid** of thread blocks.
- A **block** contains up to 1024 threads.
- Hardware executes threads in **warps** of 32.
- Built-in block and thread indices assign each thread its work.

Memory and coordination

- Blocks are scheduled onto streaming multiprocessors (SMs).
- Threads in one block cooperate through fast on-chip **shared memory**.
- `__syncthreads()` makes shared-memory loading safe.
- The CPU host chooses the grid and block dimensions at launch.

Source: Boehm (2022).

Naive SGEMM

```
__global__ void sgemm_naive(int M, int N, int K, float alpha, const float *A,
                            const float *B, float beta, float *C) {
  const uint x = blockIdx.x * blockDim.x + threadIdx.x;
  const uint y = blockIdx.y * blockDim.y + threadIdx.y;

  if (x < M && y < N) {
    float tmp = 0.0;
    for (int i = 0; i < K; ++i) {
      tmp += A[x * K + i] * B[i * N + y];
    }

    C[x * N + y] = alpha * tmp + beta * C[x * N + y];
  }
}

void run_sgemm_naive(int M, int N, int K, float alpha, float *A, float *B,
                     float beta, float *C) {
  dim3 blockDim(32, 32);                              // Common Block size.
  dim3 gridDim(CEIL_DIV(M, 32), CEIL_DIV(N, 32)); // How many blocks to cover C
  sgemm_naive<<<gridDim, blockDim>>>(M, N, K, alpha, A, B, beta, C);
}

int main() {
  // ... set up matrices and dimensions ...
  run_sgemm_naive(M, N, K, alpha, A, B, beta, C);
}
```

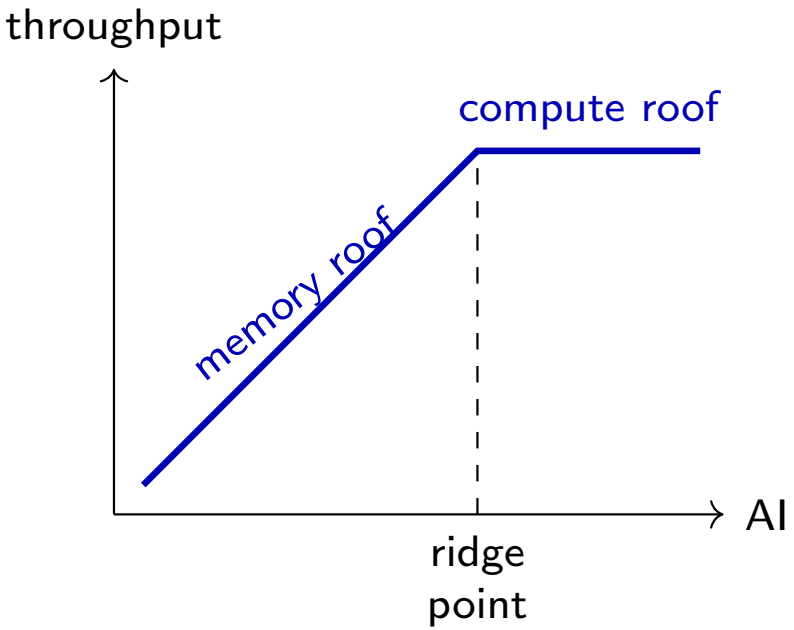
Naive SGEMM visualization

- One thread computes one element of C.
- It walks one row of A and one column of B for the full dot product.
- Neighboring threads repeatedly fetch many of the same values from global memory.

Play naive-kernel visualization

Arithmetic intensity and the Roofline Model

$$\mathrm{AI} = \frac{\mathrm{FLOPs}}{\text{bytes moved}}, \qquad P \le \min(P_{\text{peak}},\ B_{\text{mem}} \times \mathrm{AI})$$



Source: Boehm (2022).

- Low AI: performance is limited by memory bandwidth.
- High AI: performance is limited by compute throughput.
- The article's A6000 has a ridge point near $30\,\mathrm{TFLOP/s}/768\,\mathrm{GB/s} \approx 39$ FLOP/byte.
- Tiling raises AI by reusing each loaded value many times.

Basic tiled SGEMM: reuse data on chip

1. A block cooperatively loads a tile of A and a tile of B from global memory into shared memory.
2. Synchronize so every thread sees the complete tiles.
3. Each thread accumulates a partial dot product using the cached values.
4. Synchronize, advance along K, and repeat until the output tile is complete.

Why it helps

A value fetched once from global memory can serve many multiply-adds. Shared-memory cache blocking raises the article's throughput from about 2.0 to 3.0 TFLOP/s.

Source: Boehm (2022), kernels 2–3.

Basic tiled SGEMM visualization

- Watch each block load small regions of A and B together.
- The loaded tiles are reused to update an entire tile of C.
- Synchronization separates loading, computing, and advancing to the next tiles.

Play tiled-kernel visualization

Thread coarsening: reuse data in registers

- Basic tiling still assigns one output to each thread, causing frequent shared-memory loads.
- Instead, each thread accumulates multiple outputs in its private registers.
- One loaded A value can update several columns; one loaded B value can update several rows.
- A two-dimensional 8×8 register tile produces 64 outputs per thread and greatly increases arithmetic intensity.

Kernel	**Throughput**	**vs. cuBLAS**
Shared-memory tiling	3.0 TFLOP/s	12.8%
1D thread coarsening	8.5 TFLOP/s	36.5%
2D thread coarsening	16.0 TFLOP/s	68.7%

Source: Boehm (2022), kernels 3–5.

Coarsened SGEMM visualization

- Each thread now owns a small two-dimensional region of C.
- Values from shared memory are cached in registers and reused across that region.
- More outputs per thread means fewer memory operations per result.

Play coarsened-kernel visualization